An Introduction to Generative Adversarial Nets in R: The RGAN Package

by Marcel Neunhoeffer

Abstract This article introduces Generative Adversarial Nets (GAN), a powerful neural net architecture, and presents the RGAN package. RGAN makes it easy to implement GANs in R: It facilitates experimentation with different design choices for GANs, such as value functions, other network architectures, the choice of different noise distributions, the choice of optimization algorithms and update schemes, dropout during data generation and the post-processing of generated GAN samples. Furthermore, advanced users can provide customized functions for each design choice. RGAN is a lightweight package and runs on top of the torch library in R.

1 Introduction

On July 28, 2016, Yann LeCun, Deep Learning pioneer and Chief Artificial Intelligence Scientist at Facebook, was asked "What are some recent and potentially upcoming breakthroughs in deep learning?" in a Public Quora Q&A session.[1] He answered: "The most important one, in my opinion, is adversarial training (also called GAN for Generative Adversarial Networks). [...]"

So what is a Generative Adversarial Network (GAN)? As indicated in the quote above, GANs were first introduced by Goodfellow et al. (2014). They illustrate the idea with the following example: "The generative model can be thought of as analogous to a team of counterfeiters, trying to produce fake currency and use it without detection, while the discriminative model is analogous to the police, trying to detect the counterfeit currency. Competition in this game drives both teams to improve their methods until the counterfeits are indistinguishable from the genuine articles" (Goodfellow et al., 2014, p. 1).

In short, a GAN is a model that tries to learn an arbitrary joint distribution by comparison. It samples data from a proposal distribution and compares them to samples from the true distribution. Comparing the two samples (samples from the proposal distribution and samples from the true distribution) improves the model for the proposal distribution such that it generates more and more realistic samples. The basic idea of a GAN is surprisingly intuitive. At its core, a GAN is a minimax game with two competing actors: a generator that produces realistic synthetic samples from random noise and a discriminator (or critic)[2] trying to tell real from synthetic samples.

In GANs, the team of counterfeiters, the generator, is a neural network trained to produce realistic synthetic data examples from random noise. And the police, the discriminator, is a neural network to classify fake and real data. The generator network is trained to fool the discriminator network and uses the feedback of the discriminator to generate increasingly realistic fake data that should eventually be indistinguishable from the real data. At the same time, the discriminator is constantly adapting to the improving generator. Thus, the threshold where the discriminator is fooled increases along with the faking capabilities of the generator. This goes on until (in theory) an equilibrium is reached. At the equilibrium, the discriminator can no longer distinguish between real and fake samples. The generator can achieve this goal by sampling from the underlying distribution of the real data.[3]

I bring these neural networks to R with the **RGAN** package. The goal of the package is to facilitate experimentation with GANs for an audience primarily working with R. This way, GANs can be included in existing workflows, for example, for data synthesizing.

RGAN is a lightweight package that only relies on **torch** (Falbel and Luraschi, 2022). The **torch** package provides R users with native access to libtorch, the C++ backend that powers PyTorch, enabling tensor computation and deep learning capabilities directly in R without requiring Python or the **reticulate** (Ushey et al., 2022) package. This design choice makes **RGAN** particularly efficient, as **torch** offers automatic differentiation, GPU acceleration when available, and a comprehensive set of neural network modules, all with an R-native interface. By building on **torch**'s foundation, **RGAN** users benefit from fast tensor operations and seamless GPU support while maintaining a familiar R workflow, avoiding the overhead and complexity of cross-language dependencies. For applications, the focus of **RGAN** is on tabular data, yet it is easy to implement GANs for image data. Both use cases are demonstrated in section 4.

[1] As of March 3, 2023, you can find the session here: https://www.quora.com/q/quorasessionwithyannlecun.

[2] More on the difference between a discriminator and a critic can be found in section 3.

[3] A GAN is, therefore, a dynamic system where the optimization process is seeking not a minimum (or maximum) but an equilibrium.

In python, software libraries like CTGAN (Xu et al., 2019), TF-GAN, which is part of tensorflow (Abadi et al., 2015), torchgan (Pal and Das, 2021), or mimicry (Lee and Town, 2020) can serve a similar purpose as **RGAN**. Most of these libraries focus on image data.

In R, **ganGenerativeData** (Müller, 2022) makes it possible to synthesize tabular data with a GAN through **reticulate** and **tensorflow**. The focus of **ganGenerativeData** is on producing synthetic data with one specific, pre-configured GAN architecture optimized for tabular data generation. This contrasts with **RGAN**, which emphasizes flexibility and experimentation with GAN architectures.

Specifically, **RGAN** extends the functionality available in R in several key ways:

- **Native R implementation**: **RGAN** runs directly in R via **torch**, eliminating the Python dependency and **reticulate** overhead required by **ganGenerativeData**
- **Architectural flexibility**: While **ganGenerativeData** provides a fixed architecture, **RGAN** allows users to customize network architectures, create custom neural networks, and switch between architectures (e.g., fully connected for tabular data, DCGAN for images)
- **Multiple value functions**: **RGAN** implements multiple GAN variants (original, WGAN, f-WGAN) and supports custom value functions, whereas **ganGenerativeData** uses only the original GAN value function
- **Broader data support**: **RGAN** handles both tabular and image data, while **ganGenerativeData** focuses exclusively on tabular data
- **Research-oriented features**: **RGAN** provides extensive hyperparameter control, post-processing methods (DRS, PGB), and experimental features like dropout during generation

Among the Python libraries mentioned, `CTGAN` is most similar to **ganGenerativeData** in focusing on tabular data synthesis with specific architectural choices. Libraries like `TF-GAN`, `torchgan`, and `mimicry` offer similar flexibility to **RGAN** in terms of customizable architectures and multiple GAN variants, but are Python-exclusive. **RGAN** essentially brings this Python-level flexibility to R users in a native R environment.

The rest of the paper is organized as follows: In the following section A Brief Introduction to GANs, I give a more detailed introduction to GANs. In the section Designing a GAN, I introduce the different design decisions a researcher faces when developing a GAN and show how **RGAN** facilitates experimentation with these design choices. The section Illustrations offers two working examples with code: The example in Synthesizing tabular data with default settings shows how to get started with the synthesization of tabular data quickly. The example in Synthesizing images with a DCGAN architecture generates fake images with a customized neural network architecture in **RGAN**. The Summary and Outlook section provides an outlook on potential extensions of **RGAN**.

2 A brief introduction to GANs

A GAN model tries to learn an arbitrary joint distribution by comparison. Specifically, the GAN learns the joint distribution of all variables in the training data. For a dataset with variables $(x_1, x_2, \ldots, x_p)$, the GAN approximates $p(x_1, x_2, \ldots, x_p)$. For conditional GANs, this extends to learning the joint distribution of target variables given predictor variables. The GAN samples data from a proposal distribution and compares them to samples from the true distribution. Comparing the difference between the two samples improves the proposal distribution model, generating more realistic samples.

In GANs, the proposal distribution (fake data) is typically generated by a deep neural network, the generator (G). The comparison of fake and real data is made by a second deep neural network, the discriminator (D). The generator is trained to produce realistic synthetic data examples from random noise. At the same time, the discriminator has the goal of correctly distinguishing fake from real data.[4]

The generator is trained to fool the discriminator network and uses the feedback of the discriminator to generate increasingly realistic fake data that should eventually be indistinguishable from the real data. At the same time, the discriminator is constantly adapting to the improving generating abilities of the generator. Thus, the threshold where the discriminator is fooled increases along with the faking capabilities of the generator. This goes on until (in theory) an equilibrium is reached. In the classical GAN, described by the value function V in equation 1, an optimal discriminator at the equilibrium would be assigning 0.5 probability to both real and fake samples. This is where the discriminator can no longer distinguish between real and fake samples.

GANs turn a typical unsupervised learning task (learning a joint density) into a supervised learning problem (learning to distinguish between fake and real data), using the observation that it is easier to sample from p than to learn the distribution explicitly.

[4]It wants to discriminate between fake and real data, hence the name. However, novel architectures don't necessarily use classifiers to compare fake and real data. In these GANs, the second network is usually named a critic network.

Formally, this two-player minimax game can be written as:

$$\min_G \max_D V(D,G) = \min_G \max_D \mathbb{E}_{x\sim p_{\text{data}}(x)}\Big[\log D(x)\Big] + \mathbb{E}_{z\sim p_z(z)}\Big[\log(1-D(G(z)))\Big] \tag{1}$$

where $p_{data}(x)$ is the joint distribution of the real data, x is a sample from $p_{data}(x)$. The generator network $G(z)$ takes as an input z from $p_z(z)$, where z is a random sample from a probability distribution $p_z(z)$. This sample is also called a noise sample. Usually, GANs are set up to either sample z from uniform or Gaussian distributions.[5] Passing the noise sample z through G generates a sample of synthetic data, which is then fed into the discriminator D. The discriminator takes as input a set of labeled data, either real examples from $p_{data}(x)$ or generated examples from $G(z)$, and is trained to distinguish between real data and fake data. In the original GAN setup (Goodfellow et al., 2014), this is a standard binary classification problem.[6]

D is trained to maximize the probability of assigning the correct label to training examples and samples from $G(z)$. G is trained to minimize $\log(1-D(G(z)))$. Thus, the goal of the discriminator is to maximize function V, whereas the generator's goal is to minimize it. In practice, this is achieved by iteratively updating the two networks, holding the weights of the other network constant.

The equilibrium point for a GAN is achieved when G produces samples that come from the true underlying data distribution $p_{data}(x)$ and D is uncertain about the origin of the samples.

3 Designing a GAN

So far, many different GAN architectures have been proposed. Many of these GAN models are named, e.g., WGAN (Arjovsky et al., 2017), WGAN-GP (Gulrajani et al., 2017), KL-WGAN (Song and Ermon, 2020), DCGAN (Radford et al., 2016), CGAN (Mirza and Osindero, 2014), CTGAN (Xu et al., 2019), StyleGAN (Karras et al., 2019), CycleGAN (Zhu et al., 2017) or DiscoGAN (Kim et al., 2017) to name a few. Typically, these new GAN models address some areas for improvement in prior models, where most of the development focuses on generating more and more realistic images.

In this section, I first introduce the basic functionality of the RGAN package. Then, I briefly overview some of these design decisions researchers face when developing a GAN for their application. Some of these design decisions are similar to those when designing neural network models (or other machine learning models).[7]

Other decisions are unique to the setup of GAN models.[8] Generally, the interaction of all these design choices still needs to be better understood and is an active research area. Table 1 gives an overview of the most important hyperparameters and design choices supported by **RGAN**.

[5] Research suggests that other prior distributions might be preferable depending on the application. Huster et al. (2021) show that a Pareto distribution performs better than a normal distribution for skewed and heavy-tailed real data.

[6] Thus, the output layer of the discriminator uses a sigmoid function as the activation function and the standard binary cross-entropy loss can be used.

[7] For example, data pre-processing, the choice of the network architecture(s), or the choice of an optimizer with its related hyperparameters.

[8] For example, the choice of the noise distribution $p(z)$ or the GAN value function.

Table 1: Overview of Hyperparameters and Design choices and their default values in **RGAN**.

Hyperparameter	Default in RGAN	Available Options	Use Cases & Advantages	Practical Tuning Implications
Value Function	"original"	• "original": Binary cross-entropy • "wasserstein": WGAN • "f-wgan": KL-WGAN • Custom functions	• Original: Simple classification, intuitive • Wasserstein: Addresses vanishing gradients • F-WGAN: Better empirical results • Custom: Research experimentation	Affects gradient stability; Wasserstein often more stable but requires weight clipping; original may suffer from mode collapse
Network Architecture	Fully connected (2 layers, 128 neurons each	• Fully connected (customizable) • DCGAN (for images) • Custom `torch::nn_module` objects	• FC: Tabular data, quick prototyping • DCGAN: Image synthesis • Custom: Domain-specific requirements	Deeper networks = more capacity but slower training; width affects representation power; architecture should match data type
Dimension of Noise	2	Any positive integer	• Small: fewer latent dimensions • Large: more latent dimensions	Affects generator's ability to learn target distribution; should match intrinsic dimension of data
Noise Distribution	"normal"	• "normal": Standard Gaussian • "uniform": [-1, 1] • Custom functions (e.g., Pareto)	• Normal: General purpose • Uniform: DCGAN standard • Pareto: Heavy-tailed real data	Affects generator's ability to match target distribution; Pareto better for skewed data
Optimizer	Adam (`base_lr=0.0001`) `ttur_factor = 4`	• Any torch optimizer • Separate G/D optimizers • TTUR (different learning rates for G and D)	• Adam: Good default, adaptive • SGD: More control, simpler • TTUR: Stabilizes training	Higher lr = faster but unstable; TTUR factor >1 slows discriminator, preventing overfitting
Dropout	0.5 (training) FALSE (generation)	• Rate: 0.0-1.0 • eval_dropout: TRUE/FALSE	• Training: Regularization • Generation with dropout increases diversity • No dropout: More consistent outputs	Higher dropout = more regularization but may underfit; eval dropout trades consistency for diversity
Post-processing	None	• DRS (Discriminator Rejection) • PGB (Post-GAN Boosting) • None	• DRS: Quality filtering • PGB: Ensemble improvement • None: Fastest, direct output	DRS reduces sample size but improves quality; PGB requires multiple generators

Hyperparameter	Default in RGAN	Available Options	Use Cases & Advantages	Practical Tuning Implications
Batch Size	50	Any positive integer	• Small (10-50): More updates, noisy gradients • Large (128+): Stable, fewer updates	Larger = more stable gradients but slower convergence; memory constraints on GPU
Epochs	150	Any positive integer	• Few (10-50): Quick experiments • Many (150+): Better convergence	More epochs ≠ always better (can overfit discriminator); monitor convergence

In the following, I briefly describe the design choices to be made, how RGAN implements default options, and how RGAN can be used to customize all these design choices to facilitate experimentation.

3.1 The general RGAN workflow

Before discussing the various design choices and default values available when implementing a GAN and how to implement them, I first introduce the main functions and a typical workflow with the RGAN package.

The `gan_trainer()` function The core function in RGAN is `gan_trainer()`, which orchestrates the entire GAN training process. This function accepts the following key arguments:

```
gan_trainer(
  data,
  noise_dim = 2,
  noise_distribution = "normal",
  value_function = "original",
  data_type = "tabular",
  generator = NULL,
  generator_optimizer = NULL,
  discriminator = NULL,
  discriminator_optimizer = NULL,
  base_lr = 1e-04,
  ttur_factor = 4,
  weight_clipper = NULL,
  batch_size = 50,
  epochs = 150,
  plot_progress = FALSE,
  plot_interval = "epoch",
  eval_dropout = FALSE,
  synthetic_examples = 500,
  plot_dimensions = c(1, 2),
  device = "cpu"
)
```

A typical RGAN workflow Assuming tabular data that is available as a `matrix` or `data.frame` object with the name `data` in the R Session, a standard RGAN workflow consists of three main steps:

1. Data Preparation: Transform and standardize (i.e., z-standardize numerical columns and one-hot encode categorical columns) input data. The RGAN package provides the `data_transformer` class for easy transformation and back-transformation.

```
transformer <- data_transformer$new()
transformer$fit(data)
transformed_data <- transformer$transform(data)
```

2. GAN Training: Train the GAN with desired hyperparameters by calling the `gan_trainer()` function (here all hyperparameters and choices are set to default values).

```
trained_gan <- gan_trainer(transformed_data)
```

3. Synthetic Data Generation: Generate and transform synthetic samples

```
synthetic_data <- sample_synthetic_data(trained_gan,
                                        transformer)
```

3.2 Choosing a value function

Description of the design choices. An important design decision is the choice of the value function V. In the original GAN by Goodfellow et al. (2014) and described by the value function in equation 1, the value function was motivated by turning the unsupervised learning task into a binary classification problem. After all, the original GAN value function is just binary-cross entropy loss.[9]

While this is an intuitive formulation, if the discriminator is very good at distinguishing between real and fake data, it is tough to learn (i.e., update the network weights) due to vanishing gradients. This means that the gradient values are close to 0 and cannot provide meaningful information to update the weights during backpropagation.

To address this weakness, Arjovsky et al. (2017) proposed an alternative value function based on the Wasserstein distance. They use the observation that the original GAN value function essentially minimizes the Jensen-Shannon divergence between encoded fake and real data and note that using the Wasserstein distance instead has some desirable properties. This leads to the following value function:

$$V(D,G)_{||D||_L \leq 1} = \mathbb{E}_{x \sim p_{\text{data}}(x)}\Big[D(x)\Big] + \mathbb{E}_{z \sim p_z(z)}\Big[(1 - D(G(z)))\Big] \tag{2}$$

This is the value function for the so-called Wasserstein GAN or short WGAN as defined in (Arjovsky et al., 2017). Note that for the Wasserstein value function, the discriminator must be continuous with a Lipschitz constant ≤ 1, indicated by the constraint $||D||_L \leq 1$ in equation 2. The Lipschitz constraint means that the discriminator function cannot change too rapidly; formally, there exists a constant $L \leq 1$ such that $|D(x_1) - D(x_2)| \leq L \cdot |x_1 - x_2|$ for all inputs x_1, x_2. This constraint is typically enforced by clipping the discriminator's weights to a pre-specified range, e.g., $[-0.01, 0.01]$.

Work by Song and Ermon (2020) shows that f-divergences and Wasserstein GANs can be bridged. They propose a novel GAN value function that leads to better empirical results on many tasks.

A more general formulation for the value function is therefore given by

$$V(D,G) = \mathbb{E}_{x \sim p_{\text{data}}(x)}\Big[f_1(D(x))\Big] + \mathbb{E}_{z \sim p_z(z)}\Big[f_2(1 - D(G(z)))\Big] \tag{3}$$

Where in the original GAN $f_1(a) = f_2(a) = \log(a)$, in the Wasserstein GAN $f_1(a) = f_2(a) = a$ and for the GAN value function proposed by Song and Ermon (2020) $f_1(a) = a$ and $f_2(a) = wa$, where w is a weight for the discriminator scores on fake examples, calculated as $w = \exp(a)/\mathbb{E}\Big[\exp(a)\Big]$ (using the exponential function $\exp(\cdot)$).

Implementation of default design choices in RGAN. In RGAN, these three value functions are readily implemented (`original`, `wasserstein` and `f-wgan`) and can be chosen as an option, e.g., `gan_trainer(..., value_function = "wasserstein")`, the default value function is `original`.

Further customization with RGAN. Users can also provide custom value functions to work with the RGAN `gan_trainer()` function. The only requirements are that the value function takes as inputs the discriminator scores of real and fake data, works on and with torch data types, and outputs a named list with the entries `d_loss` and `g_loss`. For example, this is what the original GAN value function looks like in code:

```
GAN_value_fct <- function(real_scores, fake_scores) {
  # Compute log(1 - fake_scores) once and reuse
  log_one_minus_fake <- torch::torch_log(1 - fake_scores)
```

[9] Note that this is the same as the negative log-likelihood of the Bernoulli distribution.

```
  d_loss <-
    torch::torch_log(real_scores) + log_one_minus_fake

  d_loss <- -d_loss$mean()

  g_loss <- log_one_minus_fake$mean()

  return(list(d_loss = d_loss,
              g_loss = g_loss))

}
```

When a custom value function is passed to `gan_trainer()`, it is called at each training iteration within the training loop. The training loop follows this flow: (1) sample a minibatch of real data; (2) generate fake data by passing noise through the generator; (3) compute discriminator scores for both real and fake data (`real_scores` and `fake_scores`); (4) call the value function with these scores to obtain `d_loss` and `g_loss`; (5) backpropagate `d_loss` to update discriminator weights; (6) backpropagate `g_loss` to update generator weights. This design allows users to experiment with arbitrary training objectives while **RGAN** handles the data loading, network updates, and training orchestration.

3.3 Choosing and customizing network architectures

Description of the design choices. Parts of the network architecture, specifically the discriminator (or critic) network architecture, depend on the chosen value function. For the original GAN value function, the discriminator acts as a binary classifier outputting a probability that a sample is real. Therefore, the output layer must return values between 0 and 1, achieved by using the sigmoid activation function $\sigma(x) = 1/(1 + \exp(-x))$.[10] In contrast, for WGAN the discriminator (called a "critic" in this context) does not output a probability but rather a score measuring how "real" a sample appears. Since this score represents a distance measure rather than a probability, the output does not need to be restricted to $[0, 1]$. Therefore, a linear output layer (no activation function) is used.[11]

In general, the discriminator must have enough capacity, that is, the network's ability to represent complex functions, which is determined by factors such as the number of parameters, depth (number of layers), and width (neurons per layer). What constitutes "enough capacity" depends on the application. For image synthesization, convolutional neural network architectures achieve the best results. For time series, recurrent neural network architectures are sensible choices. For tabular data, simpler, fully connected network architectures are usually sufficient.

Implementation of default design choices in RGAN. In **RGAN**, some commonly used architectures are already implemented, e.g., DCGAN[12] (Radford et al., 2016) for image data and a GAN with fully connected neural networks for tabular data. If no networks are specified in a call to `gan_trainer` it defaults to fully connected networks for both the generator and discriminator, with two hidden layers, 128 neurons in each layer, and a dropout rate of 0.5.

Further customization with RGAN. The included fully connected network architecture can easily be customized. For example, if users want to customize the number of layers and neurons per layer, they can create custom neural networks with a call to

```
generator <- Generator(noise_dim = 2,
                       data_dim = 2,
                       hidden_units = list(256, 128, 64))
```

and

```
discriminator <- Discriminator(data_dim = 2,
                               hidden_units = list(256, 128, 64),
                               sigmoid = TRUE)
```

[10] This is one of the reasons for the vanishing gradients.

[11] For WGAN to work, however, restrictions on the discriminator weights must be imposed, either by weight clipping or a gradient penalty (e.g., in the WGAN-GP implementation (Gulrajani et al., 2017)).

[12] Short for Deep Convolutional GAN.

respectively. These networks can then be fed to `gan_trainer(..., generator = generator, discriminator = discriminator)`. Where `data_dim` needs to match the number of columns in the input dataset and `noise_dim` needs to match the dimensions of the noise distribution (more in section 3.3). The list passed to`hidden_units` specifies both the depth of the neural network and the width of each layer. For example, `list(256, 128, 64)` means that the network will have three hidden layers (determined by the length of the list), where the first hidden layer will contain 256 neurons, the second hidden layer 128, and the third hidden layer 64. For the discriminator network, users can also specify whether the output layer should have a sigmoid activation function or be a linear output layer. This choice depends on the value function chosen for a specific GAN (see section 3.1).

If users want to synthesize images and use the DCGAN architecture included in **RGAN**, they can achieve this by setting up the DCGAN networks by calling `generator <- DCGAN_Generator()` and `discriminator <- DCGAN_Discriminator()`.[13] Again, these networks can be included in the call to `gan_trainer(..., generator = generator, discriminator = discriminator)`. I describe a complete working example with the DCGAN architecture and loading images from a user-specified folder in section 4.

Furthermore, users can provide custom network architectures for the discriminator and generator networks by passing a suitable `torch::nn_module` object to the `gan_trainer` function. For the generator, that means the input to the `torch::nn_module` needs to match the dimensions of the chosen noise distribution, and the output needs to line up with the dimensions of the real data. The input must match the dimensions of the real data for the discriminator, and the output needs to be just a 1-dimensional vector with the discriminator scores.

The number of hidden layers, the depth and width of these layers, and their activation functions are hyperparameters that should be optimized for a particular application. In **RGAN**, it is easy to experiment with these hyperparameters and tailor them for a specific application.

3.4 Choosing the noise distribution

Description of the design choices. In most GAN applications, the noise distribution for the generator $p(z)$ is typically a standard normal distribution or a uniform distribution on the interval $(-1, 1)$.

However, Huster et al. (2021) show that the choice of noise distribution influences the behavior of GAN training. For example, learning skewed distributions[14] with standard noise distributions is hard. Other noise distributions, such as sampling from a Pareto distribution, could have preferable properties for different applications.

Implementation of default design choices in RGAN. This is an active area of research, and **RGAN** makes it easy to implement custom prior distributions. Users can easily choose between a `normal` distribution and a `uniform` distribution when specifying the options of `gan_trainer`.

Further customization with RGAN. On top of these default options, users can also provide their functions. For example, this is how sampling from the normal distribution is implemented:

```
normal_noise <- torch::torch_randn
```

and this function would be used as follows `gan_trainer(..., noise_distribution = normal_noise)` for training a GAN.

3.5 Choosing optimization algorithms

Description of the design choices. The optimization of GANs is notoriously hard. Research on the optimization of GANs shows (see, e.g., Heusel et al., 2017; Schaefer and Anandkumar, 2019) that gradient ascent descent (i.e., iteratively training the generator and discriminator with the same optimizer) might not be the best option.

In particular, Heusel et al. (2017) have shown that using two different learning rates for the optimization of the discriminator and generator (they call this scheme "two time-scale update rule") improves GAN training. Besides the learning rate, the concrete choice of an optimizer can also influence GAN training. The batch size, how many examples are sampled from the training data per iteration, and potentially further hyperparameters of standard optimizers (typically momentum hyperparameters) influence how quickly a GAN can learn.

[13] If no options are specified, this will initialize the default DCGAN architecture with a `noise_dim = 100` and a linear output layer for the discriminator.

[14] A typical social science example for such skewed distributions could be the distribution of income.

Implementation of default design choices in RGAN. In RGAN, it is easy to put the optimizers for the generator and discriminator on two separate time scales by setting the `ttur_factor` in `gan_trainer()`. The `ttur_factor` is a multiplier for the provided `base_lr`, so the generator will be updated with the `base_lr` and the discriminator with `ttur_factor*base_lr`. By default, RGAN uses the Adam optimizer (Kingma and Ba, 2014) as implemented in `torch::optim_adam`, and the default minibatch size is set to 50 examples per iteration.

Further customization with RGAN. RGAN facilitates experimentation with different optimizers and hyperparameter settings. Users can directly pass any torch optimizer with any self-chosen hyperparameter settings to `gan_trainer`, e.g.,

```
gan_trainer(...,
            generator_optimizer = torch::optim_sgd(g_net$parameters, lr = 0.1))
```

.

3.6 Choosing dropout during training and data generation

Description of the design choices. In neural network architectures, so-called dropout layers (Srivastava et al., 2014) can be used to regularize network training and prevent overfitting. In each training iteration, a dropout layer randomly sets a pre-specified percentage of all neurons to 0. A helpful side effect of such dropout layers in classical neural networks is that dropout can also be used to estimate the model uncertainty of neural networks (Gal and Ghahramani, 2016).

Anecdotal evidence shows that dropout layers might also help train GANs.[15] Furthermore, Isola et al. (2016) argue that dropout in the generator during data generation provides stochasticity for the synthetic data that promotes the diversity of the generated data.

Yet, a systematic evaluation of the effect of dropout in GANs on the produced synthetic data, especially on the statistical utility of tabular synthetic data, still needs to be included and warrants more research.

Implementation of default design choices in RGAN. By making it easy to experiment with dropout layers in discriminator and generator networks, RGAN can facilitate such research. When working with the default networks in `gan_trainer`, the dropout rate is set to 0.5 in both the discriminator and generator networks. By default, dropout is disabled during synthetic data generation but can be enabled by setting `gan_trainer(..., eval_dropout = TRUE)`.

Further customization with RGAN. By providing custom networks, the dropout rate during training can be adjusted. In section 4 I provide a simple example of an experiment of synthetic data generated with and without dropout during data generation.

3.7 Post-Processing GAN samples

Description of the design choices. Since convergence to equilibrium cannot be guaranteed, the samples produced after the last update need not be the best. However, several methods have been proposed to boost the samples' fidelity by sampling from multiple generators.[16] These post-processing methods leverage two empirical observations. First, the learned discriminator can assess the quality of generated examples (Azadi et al., 2019). And second, while an arbitrary generator (for example, the last generator) can produce an inadequate representation of the underlying data distribution, a mixture distribution of samples from multiple generators can improve the quality of generated examples (see, e.g., Beaulieu-Jones et al., 2019; Neunhoeffer et al., 2021). Methods include Discriminator Rejection Sampling (DRS) (Azadi et al., 2019), Metropolis-Hastings GAN (MH) (Turner et al., 2019), or Post-GAN Boosting (PGB) (Neunhoeffer et al., 2021).

[15] For example, this `https://github.com/soumith/ganhacks` list of tips for GAN training by some of the authors of influential GAN papers (Arjovsky et al., 2017; Radford et al., 2016; Denton et al., 2016; Zhao et al., 2016) mentions dropout in the generator as a means to stabilize GAN training.

[16] There are many ways to sample from multiple generators. E.g., each update step during training produces a slightly different generator, so the sequence of generators can be thought of as a distribution of generators. Or when using dropout in the generator during the generation of synthetic examples and repeating this step multiple times, this will also produce a distribution of generators. Furthermore, methods that train multiple generators in parallel also exist (e.g., Arora et al., 2017; Hoang et al., 2018).

Implementation of default design choices in RGAN. DRS and PGB are currently implemented as part of RGAN.

4 Illustrations

In this section, I provide two illustrations of how RGAN can be used. The first example shows how to quickly train a first GAN on tabular data with the default settings of RGAN, including a simple experiment of what dropout during synthetic data generation does. The second example shows that RGAN can also synthesize image data and serves as a tutorial on providing custom neural network architectures and optimizers to `gan_trainer`.

Note on training times: For the tabular data example with 1,000 samples and default settings (150 epochs, batch size 50), training completes in approximately 1–2 minutes on a modern CPU. The image synthesis example using the CelebA dataset trains for approximately 17 minutes per epoch on a GPU (e.g., GeForce RTX 2070), or approximately 7 hours per epoch on CPU.[17] Training times will vary depending on hardware, dataset size, network architecture, and hyperparameter choices.

4.1 Synthesizing tabular data with default settings

As a simple illustration to get started with RGAN, I show how to generate synthetic copies of a simple tabular data set. First, I create some toy data[18] and transform it (for numeric data, this means standardizing the data) to facilitate training.

```
data <- sample_toydata()
transformer <- data_transformer$new()
transformer$fit(data)
transformed_data <-  transformer$transform(data)
```

The real data for this example is displayed in panel (A) of figure 1. x on the x-axis is drawn from a standard normal distribution, and y on the y-axis is $x^2 + \mathcal{N}(0, 0.3)$. Panel (B) of figure 1 shows the data after transformation. The `data_transformer` applies z-standardization to numeric columns, subtracting the mean and dividing by the standard deviation. Since x is already standard normal, its scale changes minimally; y undergoes more noticeable rescaling but retains a similar visual range because the original variance was relatively small ($\approx 0.3^2$ around the quadratic mean).

Now the `gan_trainer()` can train a GAN to generate synthetic data from `transformed_data`.[19] By default, `gan_trainer` shows a simple progress bar to monitor training and estimate how long training will take.[20]

```
trained_gan <- gan_trainer(transformed_data)
```

This will set up the necessary generator, discriminator, and respective optimizers. The default value function is set to the original GAN value function described in equation 1. Then, the `gan_trainer` trains the GAN for 150 epochs using mini-batches of 50 real examples ($m = 50$) at each update step.

With the trained networks (which will be collected as part of the output of class `trained_RGAN` in the object `trained_gan`), it is then easy to sample synthetic data from the last generator using the function `sample_synthetic_data`. Note that to transform the synthetic data back to the original scale of the data, I pass the `transformer` to the `sample_synthetic_data` function. Both can be passed to the function `GAN_update_plot` to see the data alongside the synthetic data. This produces the plot in panel (A) of figure 2.

```
synthetic_data <- sample_synthetic_data(trained_gan, transformer)

GAN_update_plot(
```

[17] These timings are approximate and intended as rough guidance. Actual performance depends on system configuration.

[18] The default in `sample_toydata` is $N = 1000$.

[19] Similarly, data could be used directly. A first experiment could be to assess the convergence of the GAN when using the original data instead of the transformed data.

[20] The `gan_trainer` also provides the option to observe the intermediary output of the generator during training. If users set the option `plot_progress = TRUE`, a plot of real and synthetic data will be produced after each epoch. For cases where an epoch might take too long, or it is sufficient to look at the intermediary output in a longer interval, users can input the number of steps after which a new plot should be produced to the option `plot_interval`.

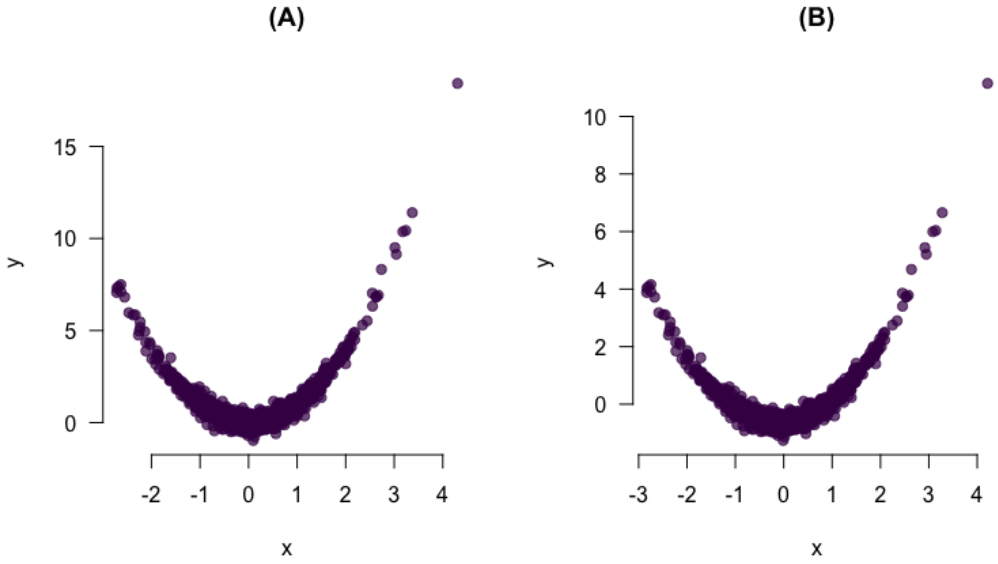

Figure 1: The real data before training the GAN. Panel (A) shows the data on the original scale and panel (B) shows the data after applying the data transformer.

```
  data = data,
  synth_data = synthetic_data,
  main = "(A)"
  )
```

Looking at the generated examples in panel (A) of figure 2 shows that while the GAN seems to approximate the functional form of the underlying data well, the produced examples have a smaller variance than the original data.

As a simple experiment, I also produce synthetic data from the same generator but with dropout during data generation. I do this by setting the `eval_dropout` setting of `trained_gan` to `TRUE`. This produces the plot in panel (B) of figure 2.

```
trained_gan$settings$eval_dropout <- TRUE

synthetic_data <- sample_synthetic_data(trained_gan, transformer)

GAN_update_plot(
  data = data,
  synth_data = synthetic_data,
  main = "(B)"
  )
```

Now the generated synthetic data in yellow looks more like the underlying real data in purple. How good the data is could be assessed using several utility metrics for synthetic data (see, e.g., Snoke et al., 2016). Note that these utility metrics are not included in **RGAN** itself. Common approaches include: (1) comparing summary statistics (means, variances, correlations) between real and synthetic data; (2) the propensity mean squared error (pMSE), which measures how well a classifier can distinguish real from synthetic data (lower values indicate better utility); and (3) downstream task performance, where models trained on synthetic data are evaluated on real test data. In R, packages such as **synthpop** (Nowok et al., 2016) provide functions for computing utility metrics. Each approach has trade-offs: summary statistics are easy to interpret but may miss complex dependencies; pMSE provides a single aggregate measure but requires training additional models; downstream evaluation is task-specific and computationally intensive but most directly measures practical utility.

4.2 The effect of different design choices/hyperparameters on GAN training

Training GANs requires careful tuning of hyperparameters, as the interplay between generator and discriminator optimization is inherently unstable. Table 1 summarizes the key hyperparameters in

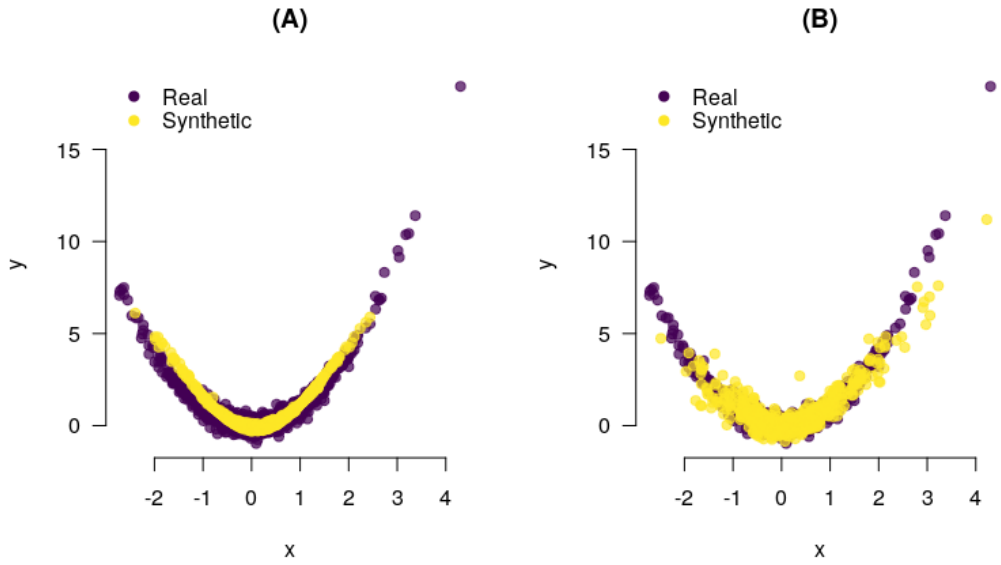

Figure 2: The synthetic data after training the GAN. Panel (A) shows the generated data without dropout during generation on the original scale. Panel (B) shows the generated data with dropout during data generation.

RGAN, but understanding their practical implications helps guide experimentation:

Learning rate: The learning rate controls how much network weights change at each update. A learning rate that is too high can cause training instability or oscillation, while one that is too low leads to slow convergence. GANs often benefit from separate learning rates for the generator and discriminator. In RGAN, the `ttur_factor` multiplies the base learning rate for the discriminator, implementing the "two time-scale update rule" (Heusel et al., 2017). A `ttur_factor > 1` (default is 4) slows down the discriminator relative to the generator, which can prevent the discriminator from becoming too strong too quickly, a common cause of vanishing gradients for the generator.

Batch size: Larger batch sizes provide more stable gradient estimates but fewer weight updates per epoch. Smaller batch sizes (e.g., 10–50) produce noisier gradients that can help escape local minima but may lead to unstable training. For tabular data, the default batch size of 50 works well for most applications. For image data, larger batch sizes (64–128) are common, though memory constraints on GPUs may limit this choice.

Network architecture: Deeper and wider networks have more capacity to represent complex distributions, but also require more data and are prone to overfitting. For tabular data with a modest number of features, two hidden layers with 128 neurons each (the RGAN default) is often sufficient. More complex data may benefit from additional layers or wider layers. The `hidden_units` parameter in `Generator()` and `Discriminator()` allows easy experimentation: `list(256, 128, 64)` creates three layers with decreasing width, which can help the network learn hierarchical representations.

Value function: The choice of value function affects gradient dynamics. The original GAN can suffer from vanishing gradients when the discriminator is too strong. The Wasserstein value function provides more informative gradients throughout training but requires weight clipping or gradient penalties. The f-WGAN value function often provides a good balance. Users should experiment with different value functions for their specific application.

4.3 Synthesizing images with a DCGAN architecture

To showcase the RGAN customization options, this illustration generates synthetic images using a DCGAN (Radford et al., 2016) architecture. The training data for this example is the CelebA dataset (Liu et al., 2015)[21] that contains 202,599 images[22] of celebrities' faces.[23]

[21]The images can be obtained from the authors' website: `https://mmlab.ie.cuhk.edu.hk/projects/CelebA.html`. Last accessed: February 03, 2026.

[22]With a batch size of 128, as in the DCGAN paper, this means the GAN is updated for 1,582 steps per epoch.

[23]Training this model on a machine with a GPU (here a GeForce RTX 2070) takes about 17 minutes per epoch. On a notebook without a GPU, one epoch takes about 7 hours.

First, I use **torchvision** to make images in a folder on my hard drive available for training the networks with **torch**. Furthermore, the DCGAN architecture expects input images to be of size 64×64 pixels. To ensure that the loaded images have the expected format, I apply some standard transformations to the raw images.

```
dataset <- torchvision::image_folder_dataset(root = "path/to/celeba",
 transform = function(x) {
 x = torchvision::transform_to_tensor(x)
 x = torchvision::transform_resize(x, size = c(64, 64))
 x = torchvision::transform_center_crop(x, c(64, 64))
 x = torchvision::transform_normalize(x, c(0.5, 0.5, 0.5), c(0.5, 0.5, 0.5))
 return(x)
 })
```

Next, I load the `DCGAN_Generator` and `DCGAN_Discriminator`, and make them available on the GPU by passing them to the `"cuda"` device.[24] The `noise_dim` (the number of input features for the generator network) is set to 100.

```
device <- "cuda"

g_net <- DCGAN_Generator(dropout_rate = 0, noise_dim = 100)$to(device = device)

d_net <- DCGAN_Discriminator(dropout_rate = 0, sigmoid = FALSE)$to(device = device)
```

Next, the optimizers for both networks can be set up. I use the same optimizers and hyperparameters as Radford et al. (2016) in the original DCGAN paper.

```
g_optim <- torch::optim_adam(g_net$parameters, lr = 0.0002, betas = c(0.5, 0.999))

d_optim <- torch::optim_adam(d_net$parameters, lr = 0.0002, betas = c(0.5, 0.999))
```

Now, these objects and a couple of additional options can be passed to `gan_trainer`. The DCGAN architecture expects the noise samples to be in a three-dimensional tensor (`noise_dim = c(100, 1, 1)`, even though the second and third dimensions are only length 1). Another important option is to set `data_type = "image"` to ensure the `gan_trainer` produces the output in the correct format.

```
trained_gan <- gan_trainer(
  data = dataset,
  noise_dim = c(100, 1, 1),
  noise_distribution = "uniform",
  data_type = "image",
  value_function = "wasserstein",
  generator = g_net,
  generator_optimizer = g_optim,
  discriminator = d_net,
  discriminator_optimizer = d_optim,
  plot_progress = TRUE,
  plot_interval = 10,
  batch_size = 128,
  synthetic_examples = 16,
  device = device,
  eval_dropout = FALSE,
  epochs = 1
  )
```

I set `plot_progress = TRUE` and `plot_interval = 10` to observe the training progress. This means 16 images[25] will be shown after every tenth update step.

In figure 3, I show 16 examples of generated faces after just one epoch of training the DCGAN with the settings as in the code above. I do not expect to generate state-of-the-art fake images after training

[24] If no GPU is available, the device should be set to `"cpu"`. If you use Apple Silicon you can also set the device to `"mps"` which would use the Apple GPU.

[25] 16 is an arbitrary number, but it is easy to show these images in a 4×4 grid.

Figure 3: Generated faces after one epoch of training the DCGAN with the parameters specified in the main text.

just for one epoch with a relatively small and simple DCGAN architecture. However, even after a few update steps with the settings described above, it should be possible to make out the outlines of faces in the generated images.

With **RGAN**, more elaborate network architectures could be provided, and training could be optimized.[26]

5 Summary and outlook

With **RGAN**, I introduce a package to facilitate training and experimentation with GANs in R. The goal is to open up this powerful neural net framework to users that primarily work in R, especially to facilitate experimentation with synthetic data from GANs from a statistical perspective.

I provide two illustrations of how **RGAN** can be used. The first illustration shows the basic

[26]Yet, achieving state-of-the-art performance can be prohibitively expensive. For example, the developers of StyleGAN2 (Karras et al., 2020), a GAN that produces high definition real looking images, transparently states the computing resources that went into the project: "The entire project, including all exploration, consumed 132 MWh of electricity, of which 0.68 MWh went into training the final FFHQ model. In total, we used about 51 single-GPU years of computation (Volta class GPU)" (8). A large amount of computing time, especially from the perspective of a statistician or social scientist. At the time of writing, the price for such a GPU, e.g., an Nvidia Tesla V100 GPU, is around $9000, i.e., to get all the experiments done in one year, you would need at least $459000 to buy the GPUs. Using cloud computing, e.g., through Amazon Web Services (AWS), you can access a computer with eight such GPUs for $12.24 per hour. To run all the experiments that (Karras et al., 2020) did, you would need 55845 hours of such a server, totaling $\approx$ $685000.

functionality of **RGAN** and how to synthesize tabular data quickly. The second illustration shows that training GANs with **RGAN** is easy to customize and synthesize images with a DCGAN architecture.

GANs have broad applications beyond the examples shown here. In statistics and data science, GANs can address class imbalance problems by generating synthetic samples of underrepresented classes, improving classifier performance on imbalanced datasets. In medical and health research, GANs enable sharing of realistic synthetic patient data for collaborative research while preserving patient privacy. This is particularly valuable in cancer research and clinical trials where real data is sensitive and scarce. The post-processing methods in **RGAN** (DRS and PGB) can help ensure that synthetic samples maintain high fidelity to real data characteristics.

Future iterations of **RGAN** will include different network architectures, for example, a CTGAN (Xu et al., 2019) inspired architecture to make the generation of mixed tabular data (i.e., data with real-valued and categorical columns) easier. Recurrent neural network (RNN) architectures could extend **RGAN** to time series data, where GANs can learn temporal dependencies. Conditional GANs (Mirza and Osindero, 2014) would allow generation of data conditioned on specific attributes, enabling targeted data augmentation. Another development area will focus on making the opacus (Yousefpour et al., 2021) python library available in **RGAN** to train GANs with formal differential privacy guarantees, providing mathematical bounds on the privacy risk of synthetic data.

The latest stable version of **RGAN** is available from CRAN; development versions can be found on `https://github.com/mneunhoe/RGAN`.[27]

Computational details

The results in this paper were obtained using R 4.5.2 with the **RGAN** 0.1.1 package, the **torch** 0.16.3 package, and the **torchvision** 0.8.0 (Falbel, 2022) package to load images into **torch**. I used the **viridis** 0.6.5 (Garnier et al., 2021) package for the color palettes in the visualizations. R itself and all packages used are available from the Comprehensive Archive Network (CRAN) at `https://CRAN.R-project.org/`.

Acknowledgments

I am grateful to Thomas Gschwend, Christian Arnold, Pia Kürzdörfer, Guido Ropers, Oliver Rittmann, and Oke Bahnsen for feedback on earlier versions of this manuscript.

References

M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin, S. Ghemawat, I. Goodfellow, A. Harp, G. Irving, M. Isard, Y. Jia, R. Jozefowicz, L. Kaiser, M. Kudlur, J. Levenberg, D. Mané, R. Monga, S. Moore, D. Murray, C. Olah, M. Schuster, J. Shlens, B. Steiner, I. Sutskever, K. Talwar, P. Tucker, V. Vanhoucke, V. Vasudevan, F. Viégas, O. Vinyals, P. Warden, M. Wattenberg, M. Wicke, Y. Yu, and X. Zheng. TensorFlow: Large-scale machine learning on heterogeneous systems, 2015. URL `https://www.tensorflow.org/`. Software available from tensorflow.org. [p6]

M. Arjovsky, S. Chintala, and L. Bottou. Wasserstein generative adversarial networks. In D. Precup and Y. W. Teh, editors, *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 214–223. PMLR, 06–11 Aug 2017. URL `https://proceedings.mlr.press/v70/arjovsky17a.html`. [p7, 10, 13]

S. Arora, R. Ge, Y. Liang, T. Ma, and Y. Zhang. Generalization and equilibrium in generative adversarial nets (GANs), 2017. URL `https://arxiv.org/abs/1703.00573`. [p13]

S. Azadi, C. Olsson, T. Darrell, I. Goodfellow, and A. Odena. Discriminator rejection sampling. In *International Conference on Learning Representations*, 2019. URL `https://openreview.net/forum?id=S1GkToR5tm`. [p13]

[27] The development version (0.2.0) on GitHub includes several additional features: differentially private GAN training via `dp_gan_trainer()` using OpenDP (The OpenDP Project, 2025) for formal privacy guarantees; CTGAN-style mode-specific normalization in the `data_transformer` for handling multi-modal distributions; enhanced post-GAN boosting with model checkpointing; and built-in evaluation metrics for assessing synthetic data quality including statistical divergence measures and privacy risk scores.

B. K. Beaulieu-Jones, Z. S. Wu, C. Williams, R. Lee, S. P. Bhavnani, J. B. Byrd, and C. S. Greene. Privacy-preserving generative deep neural networks support clinical data sharing. *Circulation: Cardiovascular Quality and Outcomes*, 12(7):e005122, 2019. doi: 10.1161/CIRCOUTCOMES.118.005122. [p13]

E. Denton, S. Gross, and R. Fergus. Semi-supervised learning with context-conditional generative adversarial networks, 2016. URL https://arxiv.org/abs/1611.06430. [p13]

D. Falbel. *torchvision: Models, Datasets and Transformations for Images*, 2022. URL https://CRAN.R-project.org/package=torchvision. R package version 0.4.1. [p19]

D. Falbel and J. Luraschi. *torch: Tensors and Neural Networks with 'GPU' Acceleration*, 2022. URL https://CRAN.R-project.org/package=torch. R package version 0.7.2. [p5]

Y. Gal and Z. Ghahramani. Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. In M. F. Balcan and K. Q. Weinberger, editors, *Proceedings of The 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 1050–1059, New York, New York, USA, 20–22 Jun 2016. PMLR. URL https://proceedings.mlr.press/v48/gal16.html. [p13]

S. Garnier, N. Ross, R. Rudis, A. onio Pedro Camargo, A. onio Pedro, M. Sciaini, and C. Scherer. *viridis - Colorblind-Friendly Color Maps for R*, 2021. URL https://sjmgarnier.github.io/viridis/. R package version 0.6.2. [p19]

I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio. Generative Adversarial Nets. *Advances in Neural Information Processing Systems 27*, pages 2672–2680, 2014. ISSN 10495258. doi: 10.1017/CBO9781139058452. URL http://papers.nips.cc/paper/5423-generative-adversarial-nets.pdf. [p5, 7, 10]

I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. C. Courville. Improved training of Wasserstein GANs. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper/2017/file/892c3b1c6dccd52936e27cbd0ff683d6-Paper.pdf. [p7, 11]

M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter. GANs trained by a two time-scale update rule converge to a local nash equilibrium. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper/2017/file/8a1d694707eb0fefe65871369074926d-Paper.pdf. [p12, 16]

Q. Hoang, T. D. Nguyen, T. Le, and D. Phung. MGAN: Training generative adversarial nets with multiple generators. In *International Conference on Learning Representations*, 2018. URL https://openreview.net/forum?id=rkmu5b0a-. [p13]

T. Huster, J. Cohen, Z. Lin, K. Chan, C. Kamhoua, N. O. Leslie, C.-Y. J. Chiang, and V. Sekar. Pareto GAN: Extending the representational power of GANs to heavy-tailed distributions. In M. Meila and T. Zhang, editors, *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 4523–4532. PMLR, 18–24 Jul 2021. URL https://proceedings.mlr.press/v139/huster21a.html. [p7, 12]

P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros. Image-to-image translation with conditional adversarial networks, 2016. URL https://arxiv.org/abs/1611.07004. [p13]

T. Karras, S. Laine, and T. Aila. A style-based generator architecture for generative adversarial networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2019. URL https://openaccess.thecvf.com/content_CVPR_2019/papers/Karras_A_Style-Based_Generator_Architecture_for_Generative_Adversarial_Networks_CVPR_2019_paper.pdf. [p7]

T. Karras, S. Laine, M. Aittala, J. Hellsten, J. Lehtinen, and T. Aila. Analyzing and improving the image quality of StyleGAN. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2020. URL https://openaccess.thecvf.com/content_CVPR_2020/papers/Karras_Analyzing_and_Improving_the_Image_Quality_of_StyleGAN_CVPR_2020_paper.pdf. [p18]

T. Kim, M. Cha, H. Kim, J. K. Lee, and J. Kim. Learning to discover cross-domain relations with generative adversarial networks. In D. Precup and Y. W. Teh, editors, *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1857–1865. PMLR, 06–11 Aug 2017. URL https://proceedings.mlr.press/v70/kim17a.html. [p7]

D. P. Kingma and J. Ba. Adam: A method for stochastic optimization, 2014. URL https://arxiv.org/abs/1412.6980. [p13]

K. S. Lee and C. Town. Mimicry: Towards the reproducibility of GAN research, 2020. URL https://arxiv.org/abs/2005.02494. [p6]

Z. Liu, P. Luo, X. Wang, and X. Tang. Deep learning face attributes in the wild. In *Proceedings of the 2015 IEEE International Conference on Computer Vision (ICCV)*, ICCV '15, pages 3730–3738, USA, 2015. IEEE Computer Society. ISBN 9781467383912. doi: 10.1109/ICCV.2015.425. URL https://doi.org/10.1109/ICCV.2015.425. [p16]

M. Mirza and S. Osindero. Conditional generative adversarial nets, 2014. URL https://arxiv.org/abs/1411.1784. [p7, 19]

W. Müller. *ganGenerativeData*, 2022. URL https://cran.r-project.org/package=ganGenerativeData. R package version 1.3.3. [p6]

M. Neunhoeffer, S. Wu, and C. Dwork. Private post-{gan} boosting. In *International Conference on Learning Representations*, 2021. URL https://openreview.net/forum?id=6isfR3JCbi. [p13]

B. Nowok, G. M. Raab, and C. Dibben. synthpop: Bespoke creation of synthetic data in R. *Journal of Statistical Software*, 74(11):1–29, 2016. doi: 10.18637/jss.v074.i11. URL https://www.jstatsoft.org/article/view/v074i11. [p15]

A. Pal and A. Das. TorchGAN: A flexible framework for GAN training and evaluation. *Journal of Open Source Software*, 6(66):2606, 2021. doi: 10.21105/joss.02606. URL https://doi.org/10.21105/joss.02606. [p6]

A. Radford, L. Metz, and S. Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks. In Y. Bengio and Y. LeCun, editors, *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*, 2016. URL http://arxiv.org/abs/1511.06434. [p7, 11, 13, 16, 17]

F. Schaefer and A. Anandkumar. Competitive gradient descent. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019. URL https://proceedings.neurips.cc/paper/2019/file/56c51a39a7c77d8084838cc920585bd0-Paper.pdf. [p12]

J. Snoke, G. Raab, B. Nowok, C. Dibben, and A. Slavkovic. General and specific utility measures for synthetic data, 2016. URL https://arxiv.org/abs/1604.06651. [p15]

J. Song and S. Ermon. Bridging the gap between f-GANs and Wasserstein GANs. In H. D. III and A. Singh, editors, *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 9078–9087. PMLR, 13–18 Jul 2020. URL https://proceedings.mlr.press/v119/song20a.html. [p7, 10]

N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958, 2014. URL http://jmlr.org/papers/v15/srivastava14a.html. [p13]

The OpenDP Project. *opendp: R Bindings for the OpenDP Library*, 2025. URL https://opendp.org/. R package version 0.14.1. [p19]

R. Turner, J. Hung, E. Frank, Y. Saatchi, and J. Yosinski. Metropolis-Hastings generative adversarial networks. In K. Chaudhuri and R. Salakhutdinov, editors, *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 6345–6353. PMLR, 09–15 Jun 2019. URL https://proceedings.mlr.press/v97/turner19a.html. [p13]

K. Ushey, J. Allaire, and Y. Tang. *reticulate: Interface to 'Python'*, 2022. URL https://CRAN.R-project.org/package=reticulate. R package version 1.24. [p5]

L. Xu, M. Skoularidou, A. Cuesta-Infante, and K. Veeramachaneni. Modeling tabular data using conditional GAN. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019. URL https://proceedings.neurips.cc/paper/2019/file/254ed7d2de3b23ab10936522dd547b78-Paper.pdf. [p6, 7, 19]

A. Yousefpour, I. Shilov, A. Sablayrolles, D. Testuggine, K. Prasad, M. Malek, J. Nguyen, S. Ghosh, A. Bharadwaj, J. Zhao, G. Cormode, and I. Mironov. Opacus: User-friendly differential privacy library in PyTorch. *arXiv preprint arXiv:2109.12298*, 2021. URL https://arxiv.org/abs/2109.12298. [p19]

J. Zhao, M. Mathieu, and Y. LeCun. Energy-based generative adversarial network, 2016. URL https://arxiv.org/abs/1609.03126. [p13]

J.-Y. Zhu, T. Park, P. Isola, and A. A. Efros. Unpaired image-to-image translation using cycle-consistent adversarial networks. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, Oct 2017. URL https://openaccess.thecvf.com/content_ICCV_2017/papers/Zhu_Unpaired_Image-To-Image_Translation_ICCV_2017_paper.pdf. [p7]

Marcel Neunhoeffer
Ludwig-Maximilians-Universität München
Social Data Science and AI Lab
Department of Statistics
Munich, Germany

Institute for Employment Research
Statistical Methods Research Department
Nuremberg, Germany
https://www.marcel-neunhoeffer.com
ORCiD: 0000-0002-9137-5785
marcel@marcel-neunhoeffer.com